

HiDiTingV100 lwIP

调测指南

文档版本 00B01
发布日期 2025-06-05

前言

概述

本文档详细的描述了 lwIP 组件的适配调试，组件所用开源版本目前为 v2.1.3，其中支持 TCP、UDP、DHCP 等协议。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 软件开发工程师
- 技术支持工程师
- 软件测试工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 lwIP 的适配.....	1
2 线程创建.....	2
3 南向接口适配	5
4 北向接口适配	7
5 目录说明.....	9
6 必要操作.....	11
7 调试及优化.....	12

插图目录

图 1-1 SOCKET API 整体流程	1
图 2-1 lwIP 线程	2
图 2-2 tcpip_thread 线程	3
图 2-3 CMakeLists.txt	4
图 2-4 config.py	4
图 3-1 IP 地址、网关和网络掩码	5
图 4-1 客户端和服务端线程	7
图 4-2 申请套接字	7
图 4-3 设置服务器地址信息	8
图 4-4 发送数据	8
图 4-5 绑定 socket	8
图 4-6 接收数据	8
图 5-1 适配层目录	9
图 6-1 组件库选项	11
图 7-1 lwip_main 任务	12
图 7-2 配置项	13

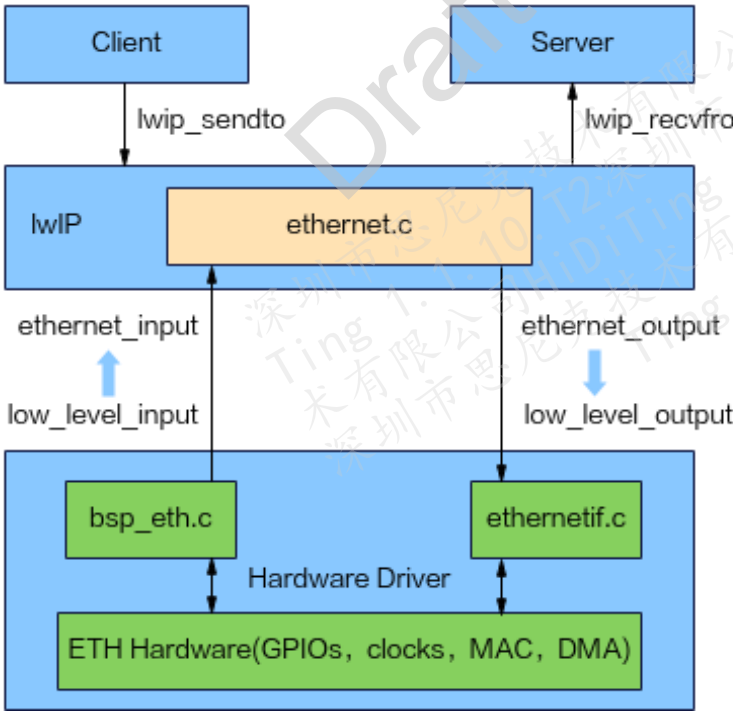
表格目录

表 5-1 常用配置宏 10

1 lwIP 的适配

lwIP 的适配主要是南北向接口的适配，南向接口主要是驱动的注册；北向接口 lwIP 提供了三种编程接口，分别为 RAW/Callback API、NETCONN API、SOCKET API，它们的易用性从左到右依次提高，而执行效率从左到右依次降低，本文主要基于 SOCKET API 进行适配，流程如图 1-1 所示。

图1-1 SOCKET API 整体流程



2 线程创建

主线程为 app_main 线程，lwIP 另起一个子线程如图 2-1 所示中的 lwIP_main。

图2-1 lwIP 线程

```
app_task_definition_t g_app_tasks[] = {  
    { { "app", 0, NULL, 0, NULL, APP_MAIN_STACK_SIZE, TASK_PRIORITY_APP, 0, 0 },  
      (osThreadFunc_t)app_main, NULL, NULL, USE_DTCM_MEM },  
#if SUPPORT_LWIP  
    { { "lwip", 0, NULL, 0, NULL, LWIP_MAIN_STACK_SIZE, TASK_PRIORITY_BRANDY_APP, 0, 0 },  
      (osThreadFunc_t)lwip_main, NULL, NULL, USE_DTCM_MEM },  
#endif  
};
```

在 lwIP 线程中调用 tcpip_init 初始化网络协议栈，并在其中创建一个 tcpip_mbox 邮箱，邮箱的大小是 TCPIP_MBOX_SIZE，用于接收从底层或者上层传递过来的消息，还会创建 tcpip_thread 线程并调用协议栈初始化操作（创建邮箱、请求互斥锁等），如图 2-2 所示。

图2-2 tcpip_thread 线程

```
static void
tcpip_thread(void *arg)
{
    struct tcpip_msg *msg;
    LWIP_UNUSED_ARG(arg);

    LWIP_MARK_TCPIP_THREAD();

    LOCK_TCPIP_CORE();
    if (tcpip_init_done != NULL) {
        tcpip_init_done(tcpip_init_done_arg);
    }

    while (1) {                                /* MAIN Loop */
        LWIP_TCPIP_THREAD_ALIVE();
        /* wait for a message, timeouts are processed while waiting */
        TCPIP_MBOX_FETCH(&tcpip_mbox, (void **)&msg);
        if (msg == NULL) {
            LWIP_DEBUGF(TCPIP_DEBUG, ("tcpip_thread: invalid message: NULL\n"));
            LWIP_ASSERT("tcpip_thread: invalid message", 0);
            continue;
        }
        tcpip_thread_handle_msg(msg);
    }
}
```

- 消息处理：lwIP 将函数 tcpip_timeouts_mbox_fetch() 定义为带宏 TCPIP_MBOX_FETCH，所以在这里就是等待消息并且处理超时事件，如果没有等到消息就继续等待，等待到消息就对消息进行处理（tcpip_thread_handle_msg 函数）。
- 编译脚本配置 lwIP 宏和组件：
编译脚本是 “/build/config/target_config/brandy/config.py”。
具体改动如图 2-4 所示，根据不同的编译选项，修改相应选项的配置，以现有 liteos 和 freertos 选项为例，分别用的 brandy-native-js、brandy-target5 版本，即在 defines 和 ram_component 中增加相应内容。
- 组件创建路径：在 “/open_source/openharmony/third_party/lwIP” 的 CMakeLists.txt 文件。

图2-3 CMakeLists.txt

```
set(COMPONENT_NAME "lwip")

set(CMAKE_LWIP_SOURCE_DIR
    ${CMAKE_CURRENT_SOURCE_DIR}/lwip_v2.1.3)
set(LWIP_DIR ${CMAKE_CURRENT_SOURCE_DIR}/lwip_v2.1.3)
```

图2-4 config.py

```
'brandy-native-js': {
    'board': 'evb',
    'base_target_name': 'target_standard_brandy_application_template',
    'defines': ['PRE_ASIC', 'BRANDY_PRODUCT_EVb4', 'VERSION_STANDARD', '__LITEOS__', 'CONFIG_OTA_UPDATE_SUPPORT',
        'ALL_SOURCE', 'SUPPORT_CXX', 'I2C_SLAVE_REG_ADDR_4BYTE', 'CONFIG_PSRAM_SUPPORT', 'CONFIG_EFUSE_READ_TO_RAM',
        'TARGET_CHIP_BRANDYv1', 'BRANDY_CHIP_V100v1', 'CFG_DRIVERS_NANDFLASH', 'CONFIG_ZDIAG_NV_SUPPORT',
        'CONFIG_ZDIAG_AUDIO_PROC_SUPPORT', 'CONFIG_ZDIAG_AUDIO_DUMP_SUPPORT', 'CONFIG_ZDIAG_AUDIO_PROBE_SUPPORT', 'CONFIG_SEA_PHS_SUPPORT', 'CONFIG_SUPPORT_BLE', 'SUPPORT_BREDR', 'ENABLE_UKIT', 'SUPPORT_GPU_PPEG', 'SUPPORT_GPU_GMMU', 'SUPPORT_GPU_OPENVG',
        'SN_UART_DEBUG', 'SAVE_EXC_INFO', 'HASH_MEM_COPY', 'JS_ENABLE', 'CONFIG_LOW_POWER_TEST', 'SUPPORT_OHOSFWK',
        'ENABLE_ECC', 'MIPI_ULPS_SUPPORT', 'SUPPORT_ALIPAY_SEC', 'SUPPORT_RC_CALIBRATION', 'CONFIG_LWIP_LOWPWR', 'SUPPORT_LWIP'],
    'ram_component': ['algorithm', 'dfx_port_brandy', 'dfx_update', 'dfx_nv', 'osal', 'arch_port', 'testsuite',
        'liteos_port', 'lcd', 'qspi_display', 'dfx_file_operation',
        'psram', 'hal_mipi', 'mipi_tx', 'non_os', 'touch',
        'ulp_aon', 'hal_l2ram', 'bt_manager_at_service',
        'x_dpai', 'x_vfs', 'x_disk', 'x_fat', 'drv_mmc', 'fs_yaffs2', 'x_vfs_private',
        'pm_service', 'pm_brandy', 'cmn_header', 'at_cmd', 'graphic_at_service', 'app_at_service', 'ohosfwk_at_service',
        'audio_proc', 'audio_dump', 'audio_probe', 'power_manager', 'gpu_proc', 'bts_header', 'lwip', 'systemview'],
    'ram_component_set': ['bgh', 'media', 'gpu', 'graphic_uikit', 'bgh_audio', 'dfx_set', 'ohos_set', 'wearable_set', 'alipay_set', 'msg_center_service'],
    'dsp_version': 'max',
    'packet': True,
    'fs_image': False
},
```

3 南向接口适配

步骤 1 构建网卡驱动注册环境：构建 IP 地址、网关和网络掩码。

图3-1 IP 地址、网关和网络掩码

```
IP4_ADDR(&ip, IP_ADDR0, IP_ADDR1, IP_ADDR2, IP_ADDR3);  
IP4_ADDR(&gw, GW_ADDR0, GW_ADDR1, GW_ADDR2, GW_ADDR3);  
IP4_ADDR(&netmask, NETMASK_ADDR0, NETMASK_ADDR1, NETMASK_ADDR2, NETMASK_ADDR3);
```

说明

此外还需配置 MAC 地址。

步骤 2 初始化 lwIP 协议栈，调用 netif_add 接口（lwIP 社区开源框架）将其加入 netif 链表（挂载到链表上是使用的前提），根据 IP 协议 IPV4/IPV6 解析地址，初始化 netif 网卡结构，配置网卡各项基础参数，更改网络接口的 IP 地址配置（包括网络掩码和默认网关），在 0~254 范围内分配一个唯一的 netif 号（接口索引为 num+1，假如分配的号为 254，那么接口索引就是 0），最后在接口上启动 IGMP 处理。

步骤 3 调用 netif_set_default（lwIP 社区开源框架）切换默认路由。

将网络接口设置为默认网络接口，输出所有找不到具体路由的报文。

步骤 4 询问链路正常状态。

步骤 5 调用 netif_set_up 函数（lwIP 社区开源框架）开始运行。

设置回调参数，发布报文，协议分流等。

----结束

总结就是开发板上 eth 接收完数据后产生一个中断，然后释放一个信号量通知网卡接收线程去处理这些接收的数据，然后将这些数据封装成消息，投递到 tcpip_mbox 邮箱中，lwIP 内核线程得到此消息，就对消息进行解析，根据消息中数据包类型进行处理。

Draft

4 北向接口适配

创建 client 和 server 作为系统的起始和结果终端收发报文。

图4-1 客户端和服务端线程

```
void client_init(void) {  
    printf("__enter client_init function__\n");  
    sys_thread_new("server", udp_server, NULL, 4096, 4);  
    sys_thread_new("client", udp_client, NULL, 4096, 4);  
}
```

说明

线程空间要给充足，最少为 4KB。

以下以 Socket API 为例。

客户端正常进行数据发送（以 UDP 为例）

创建 UDP 套接字，也就是调用 lwIP socket 函数（开源框架，本质上其实就是对 netconn 的封装）向内核申请一个套接字，如图 4-2 所示。

图4-2 申请套接字

```
// 创建 UDP 套接字  
sockfd = socket(AF_INET, SOCK_DGRAM, 0);  
if (sockfd == -1) {  
    perror("Error creating socket");  
    exit(EXIT_FAILURE);  
}
```

设置服务器地址信息如图 4-3 所示，调用 lwIP_sendto（lwIP 社区开源框架，如图 4-4）接口发送数据，进入 lwIP 协议栈。

说明

lwIP_sendto 主要是用于 UDP 协议传输数据中，它向另一端的 UDP 主机发送一个 UDP 报文，本质上是对 netconn_send()函数的封装。

图4-3 设置服务器地址信息

```
// 设置服务器地址信息
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_port = lwip_htons(SEND_PORT);
if (inet_aton(SERVER_IP, &server_addr.sin_addr) == 0) {
    perror("Invalid server IP address");
    close(sockfd);
    exit(EXIT_FAILURE);
}
```

图4-4 发送数据

```
if (sendto(sockfd, message, strlen(message), 0, (struct sockaddr *)&server_addr, sizeof(server_addr)) == -1) {
    perror("Error sending data");
    close(sockfd);
    exit(EXIT_FAILURE);
}
```

服务端接收数据

创建 UDP 套接字，设置服务器地址信息，绑定套接字（用于服务器端绑定套接字与网卡信息，如图 4-5 所示），循环调用 recvfrom（开源框架，如图 4-6 所示）接口接收数据。

图4-5 绑定 socket

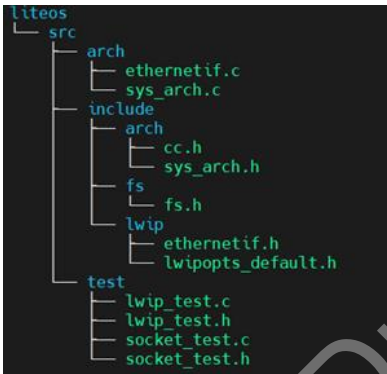
```
// 绑定套接字
if (bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) == -1) {
    perror("Error binding socket");
    close(sockfd);
    exit(EXIT_FAILURE);
}
```

图4-6 接收数据

```
// 接收数据
ssize_t bytes_received = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&client_addr, &client_len);
if (bytes_received == -1) {
    perror("Error receiving data");
    continue;
}
```

5 目录说明

图5-1 适配层目录



- sys_arch.c 为邮箱、线程创建、互斥锁、信号量、内核时间获取等功能接口实现。sys_arch.h 展示其接口。
- fs.h 文件系统适配文件。
- ethernetif.c 提供了 netif 访问各种不同的网卡，每个网卡有不同的实现方式，初始化硬件和数据包传输等操作。

说明

ethernetif.c 文件中的函数为与硬件打交道的底层函数，当有数据需要通过网卡接收或者发送数据的时候就会被调用，经过 lwIP 协议栈内部进行处理后，从应用层就能得到数据或者可以发送数据。

以下就是具体实现：

- low_level_init()为网卡初始化函数，它主要完成网卡的复位及参数初始化，根据实际的网卡属性配置 netif 中与网卡相关的字段，例如网卡的 MAC 地址、长度，最大发送单元等。

- low_level_output()函数为网卡的发送函数，它主要将内核的数据包发送出去，数据包采用 pbuf 数据结构进行描述，该数据结构是一个比较复杂的数据结构。
- low_level_input()函数为网卡的数据接收函数，该函数会接收一个数据包，为了内核易于对数据包的管理，该函数必须将接收的数据封装成 pbuf 的形式。

ethernetif.h 展示网卡实现接口。

- lwipopts_default.h 为默认配置信息，一切外部未定义或重复定义宏或变量等在此定义或进行覆盖，比如与社区源码（如 lwIP_v2.1.3）相关宏的配置选项取舍和设定，常用见[表 5-1](#)。

表5-1 常用配置宏

配置项	说明
NO_SYS	有无操作系统，有则置为 0。
LWIP_SOCKET	支持 socket 接口（需有操作系统）。
LWIP_CONN	支持 NETCONN 接口，（需有操作系统）。
LWIP_NETIF_API	支持 netif API。
TCPIP_MBOX_SIZE	定义邮箱大小。
LWIP_NUM_SOCKETS_MAX	最大支持套接字数。
MEM_LIBC_MALLOC	分配内存方式，置为 1，则使用此库函数（malloc、free）等。

- 与 lwIP_adapter 同级的 lwIP_v2.1.3 则为开源中心框架。
- test 目录内容则为示例程序，仅作参考。

6 必要操作

根据目标组件，适配 liteos 和 freertos 不同系统 API，不同系统在线程、锁、邮箱、互斥体等方面实现不同，确认是否存在相应的目标组件库。例如图 6-1 所示适配不同系统。

图6-1 组件库选项

```
if("liteos_207_1_0" IN_LIST TARGET_COMPONENT)
    set(CMAKE_LWIP_ADAPTER_DIR
        ${CMAKE_CURRENT_SOURCE_DIR}/lwip_adapter/liteos)
elseif("liteos_207_0_0" IN_LIST TARGET_COMPONENT)
    set(CMAKE_LWIP_ADAPTER_DIR
        ${CMAKE_CURRENT_SOURCE_DIR}/lwip_adapter/liteos)
elseif("freertos" IN_LIST TARGET_COMPONENT)
    set(CMAKE_LWIP_ADAPTER_DIR
        ${CMAKE_CURRENT_SOURCE_DIR}/lwip_adapter/freertos)
endif()
```

7 调试及优化

lwIP 协议栈相关功能默认为关闭状态，需打开。在

“/build/config/target_config/brandy/config.py” 和 “config.py.srcrelease” 中，编译目标（例如 brandy-native-js）要添加 SUPPORT_LWIP 宏，以打开 lwIP 特性。具体启动部分操作如图所示。

- tcpip_init 函数初始化 lwip 各模块（包括定时器模块），开启 tcpip_thread 线程（核心业务为不断循环获取并处理邮箱消息）。
- lwip_test_init 函数为南向接口用例，可开启南向 netif 等相关功能【示例，仅供参考】。
- client_init 函数为北向接口用例，通过两个 client 和 server 子线程作为两个数据收发终端，运行整个系统【示例，仅供参考】。

图7-1 lwip_main 任务

```
#if SUPPORT_LWIP
static void lwip_main(void)
{
    PRINT("lwip_main run start."NEWLINE);
    tcpip_init(NULL, NULL);
    lwip_test_init();
    client_init();
}
#endif
```

在 lwipopts.h 文件中配置 lwIP 的参数，具体见图 7-2。lwIP 管理的内存建议稍多分配，而当发送数据是存储在 ROM 或者静态存储区时，还要将 MEMP_NUM_PBUF 宏定义数据做进一步提升，发送缓冲区大小和发送缓冲区队列长度决定了发送速度的大小，根据不同需求进行配置，并且需要不断调试；而对于接收数据的配置，应该配置 TCP 缓冲队列中的报文段数量与 TCP 接收窗口大小，特别是接收窗口的大小，会直接影响数据的接收速度。

图7-2 配置项

```
#ifndef LWIP_NUM_SOCKETS_MAX
#define LWIP_NUM_SOCKETS_MAX 9
#endif

#ifndef DEFAULT_LWIP_NUM_SOCKETS
#define DEFAULT_LWIP_NUM_SOCKETS 12
#endif /* DEFAULT_LWIP_NUM_SOCKETS */

#ifndef MEMP_NUM_PBUF
#define MEMP_NUM_PBUF 8
#endif

#ifndef MEMP_NUM_RAW_PCB
#define MEMP_NUM_RAW_PCB 1
#endif

#ifndef MEMP_NUM_TCPIP_MSG_API
#define MEMP_NUM_TCPIP_MSG_API 16
#endif

#ifndef MEMP_NUM_TCPIP_MSG_INPKT
#define MEMP_NUM_TCPIP_MSG_INPKT 32
#endif

#ifndef MEMP_NUM_NETCONN
#define MEMP_NUM_NETCONN LWIP_CONFIG_NUM_SOCKETS
#endif
```